University of Geneva

Faculty of Science

Exercise #4

Introduction to Computational Finance
14X030

Michel Jean Joseph Donnet

2026-03-18

Contents

1 Time average

You can use the following snippet of Python code to generate a time series:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1)
n = 1000
ts = 100 + np.cumsum(np.random.uniform(-1, 1, n))
```

1. Code your own implementation of:
   - A simple moving average (SMA)

```
def SMA(ts: np.ndarray, N: int) -> np.ndarray:
    index = np.arange(len(ts))
    convolution_index = index[N // 2 : -N // 2 + 1]
    return convolution_index, np.convolve(ts, np.ones(N) / N,
mode="valid")


plt.figure()
plt.title("Simple Moving Average")
plt.plot(np.arange(n), ts, label="Time series")
plt.plot(*SMA(ts, N=10), label="SMA")
plt.legend()
plt.show()
```

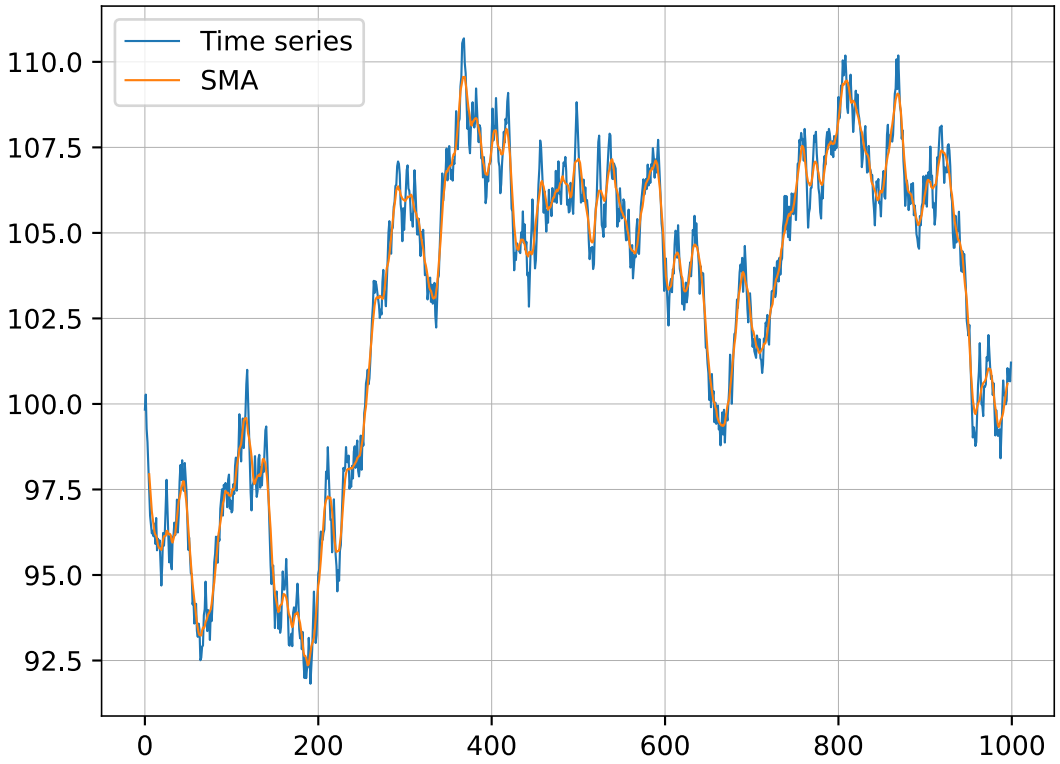


Figure 1: Simple Moving Average

- An exponential moving average (EMA)

```python
def EMA(ts: np.ndarray, alpha: float) -> np.ndarray:
    EMA = [ts[0]]
    for i in range(1, len(ts)):
        EMA.append(alpha * ts[i] + (1 - alpha) * EMA[-1])
    return np.array(EMA)


plt.figure()
plt.title("Exponential Moving Average")
plt.plot(np.arange(n), ts, label="Time series")
plt.plot(np.arange(n), EMA(ts, alpha=0.5), label="EMA")
plt.legend()
plt.show()
```

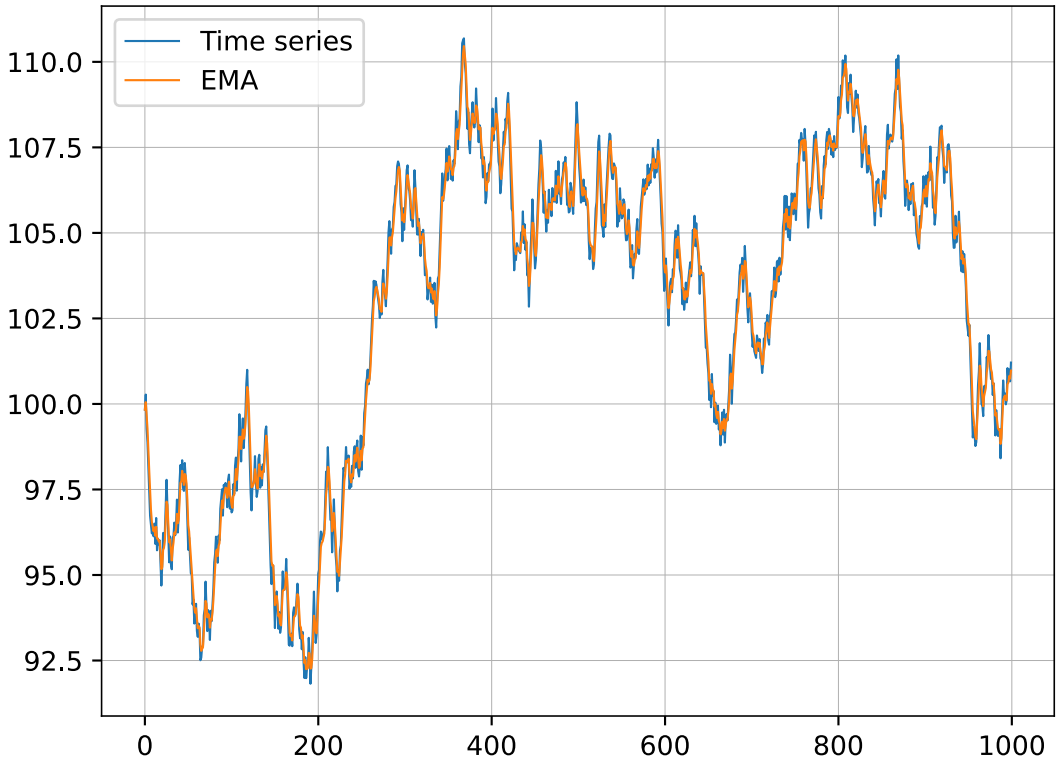


Figure 2: Exponential Moving Average

Make sure that $\text{SMA}(\text{ts}, N = 1) = \text{EMA}(\text{ts}, \alpha = 1) = \text{ts}$.

```
assert np.array_equal(ts, SMA(ts, N=1)[1]) and np.array_equal(
    ts, EMA(ts, alpha=1)
), "SMA(ts, N = 1) must be equal to EMA(ts, alpha=1) and ts !"
```

2. For the following, you can use a library if you did not succeed to implement your own moving averages. On the same graph, plot the nine following curves:
   - The original time series `ts`
   - $\left\{\text{SMA}(\text{ts}, N), \text{EMA}\left(\text{ts}, \alpha = \frac{2}{N+1}\right), \forall N \in \{10, 50, 100, 200\}\right\}$

```
plt.figure()
plt.title("Time series, simple moving average and exponential moving
average")
plt.plot(np.arange(n), ts, label="Time series")
for N in [10, 50, 100, 200]:
    plt.plot(*SMA(ts, N), label=f"SMA(ts, N={N})")
    plt.plot(np.arange(n), EMA(ts, 2 / (N + 1)), label=f"EMA(ts, alpha=2/
({N} + 1))")
plt.legend()
plt.show()
```

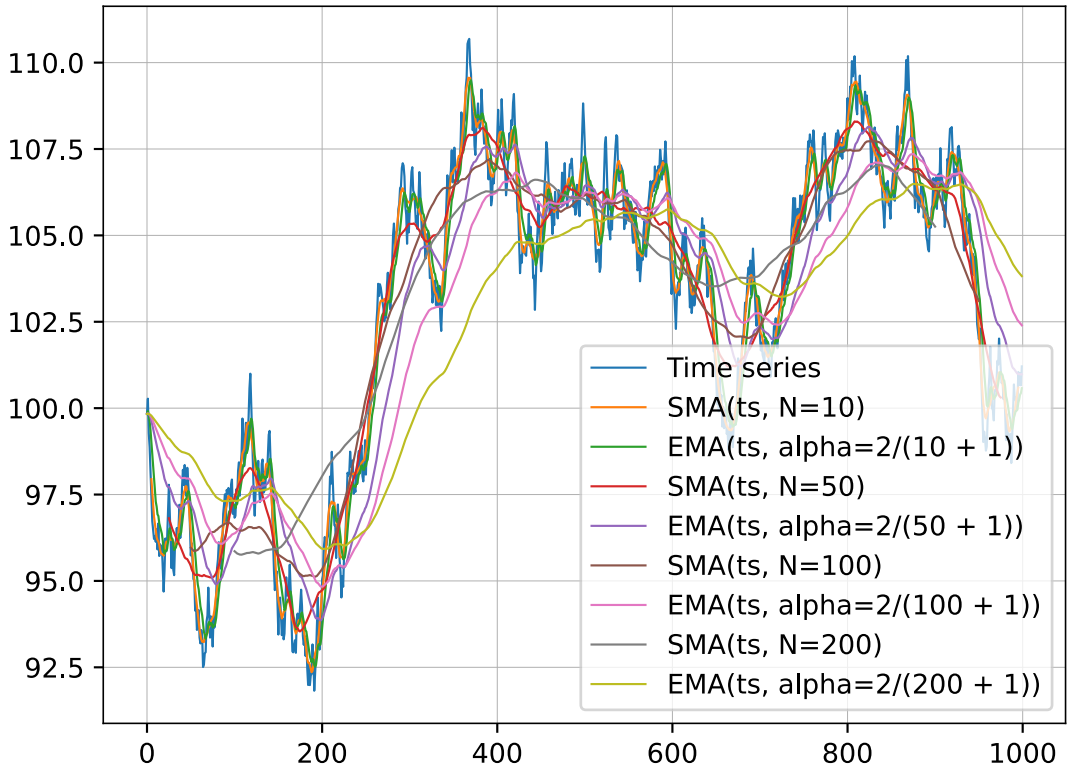


Figure 3: Time series, simple moving average and exponential moving average

What can you say of SMA vs EMA? How does the fit and the lag relate to N?

I have the impression that the EMA is always smaller than the SMA… Indeed, for all values of N, the SMA curve is over the corresponding EMA curve.

If we look at the SMA curves, they seems to be smoother than the EMA curves for a given N, meaning the EMA curves are more able to catch small differences.

Finally, if we look at for instance the orange and the green curves of SMA and EMA, we can see that the EMA seems to be a little bit shifted to the left compared to the SMA curve. And the curve that seems to match the best the picks from thte original series is the EMA curve.

So compared to the SMA, the EMA seems to take more into account the small variations of the curve. This metrics seems equaly to be more precise for the location of the picks. However, the EMA is always bellow the SMA curve, which denotes the fondamental difference between them.

3. On the same graph, plot the four following curves:
   - The original time series `ts`
   - $\{\text{EMA}(\text{ts}, \alpha), \forall \alpha \in \{0.01, 0.1, 0.5\}\}$

```
plt.figure()
plt.title("Time series and exponential moving average")
plt.plot(np.arange(n), ts, label="Time series")
for alpha in [0.01, 0.1, 0.5]:
```

```
    plt.plot(np.arange(n), EMA(ts, alpha), label=f"EMA(ts,
alpha={alpha})")
plt.legend()
plt.show()
```

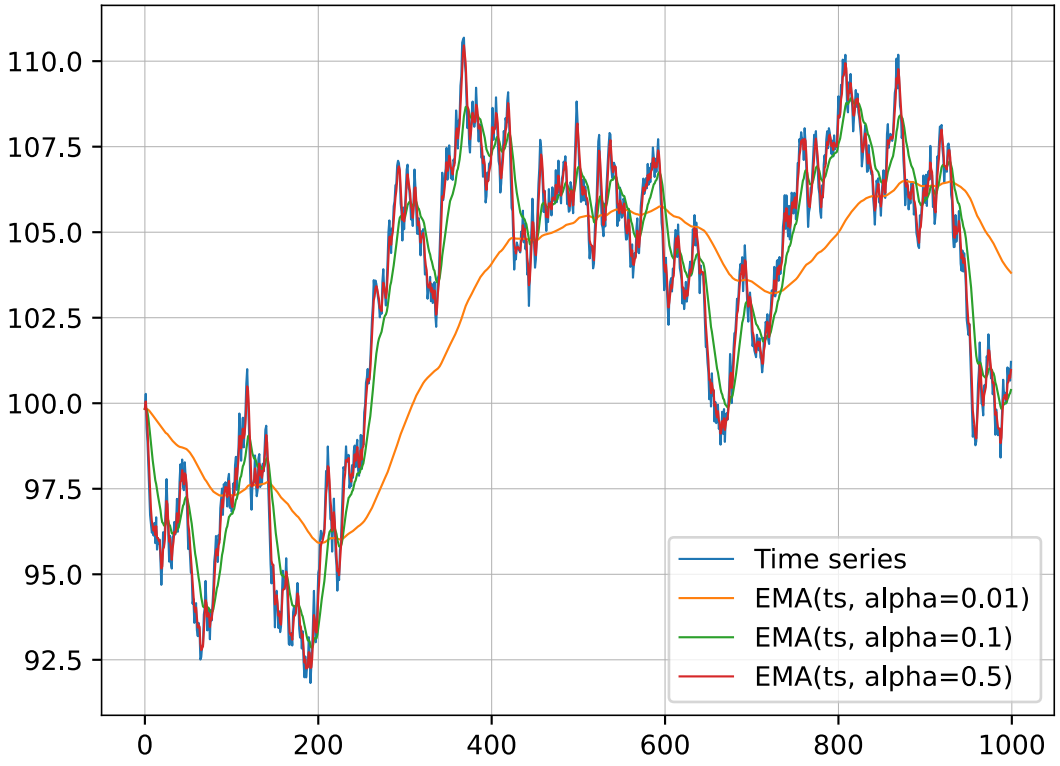


Figure 4: Time series and exponential moving average

What do you observe when α varies?

When α is big, the EMA is close to the original curve, like shown by the red curve that follows the time series curve.

However, when α is small, the EMA tend to smooth the time series since when $\alpha = 0.01$, the EMA curve is a very smooth version of the time series that don't take account of all the small variations.

So the parameter α allows to set the right level of details depending on the needs.

2 Scaling law

For this exercise, load the data provided in TP_03 (EUR/USD, tick-by-tick, from January 1st to March 1st 2012).

1. Plot the time series of mid-price against true time.
2. On the same graph, plot the directional changes for $\delta = 0.01$.

```python
import matplotlib.dates as mdates
import datetime

data = np.loadtxt("eur_usd_20120101_20120301.txt")
dates = np.array([datetime.datetime.fromtimestamp(x) for x in data[:, 0]])
bids = data[:, 1]
asks = data[:, 2]


def directional_changes(
    mid_price: np.ndarray, delta: float = 0.01
) -> tuple[np.ndarray, np.ndarray]:
    index = []
    values = []
    mode = "up"
    x_ext = mid_price[0]
    for i in range(len(mid_price)):
        x = mid_price[i]
        if mode == "up":
            if x < x_ext:
                x_ext = x
            elif (x - x_ext) / x_ext >= delta:
                x_ext = x
                mode = "down"
                values.append(x)
                index.append(i)
        else:
            if x > x_ext:
                x_ext = x
            elif (x_ext - x) / x_ext >= delta:
                x_ext = x
                mode = "up"
                values.append(x)
                index.append(i)

    index = np.array(index).astype(int)
    values = np.array(values)
    return index, values


mid_price = (asks + bids) / 2

delta = 0.01
idx_changes, changes = directional_changes(mid_price, delta=delta)

plt.figure()
plt.title("Time series of mid-price against true time")
plt.plot(dates, (asks + bids) / 2, label="Middle price")
plt.scatter(
    dates[idx_changes],
    changes,
    color="magenta",
    label=f"Directional changes with $\\delta={delta}$",
```

```
)
plt.gca().xaxis.set_major_locator(mdates.WeekdayLocator())
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%b %d %Y"))
plt.gcf().autofmt_xdate()
plt.ylabel("Price")
plt.legend()
plt.show()
```

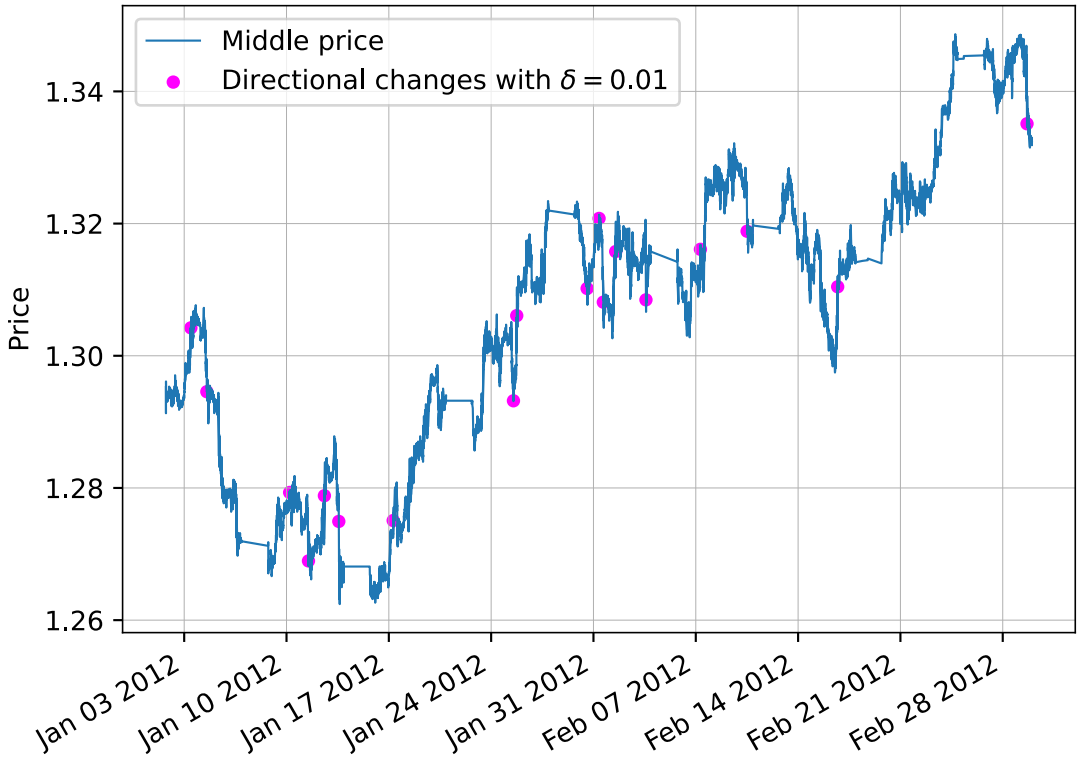


Figure 5: Time series of mid-price against true time

3. Draw a log-log plot of the number of directional changes as a function of the scale δ for values of $\delta \in [10^{-5}, 10^{-2}]$. What do you observe?

```
deltas = np.linspace(1e-5, 1e-2, 10)
counts = np.zeros_like(deltas)
for i in range(deltas.size):
    idx, _ = directional_changes(mid_price, delta=deltas[i])
    counts[i] = idx.size

plt.figure()
plt.title("Number of directional changes according to the scale $\\delta$")
plt.scatter(deltas, counts)
plt.xlabel("$\\delta$")
plt.ylabel("Number of directional changes")
plt.xscale("log")
plt.yscale("log")
plt.show()
```

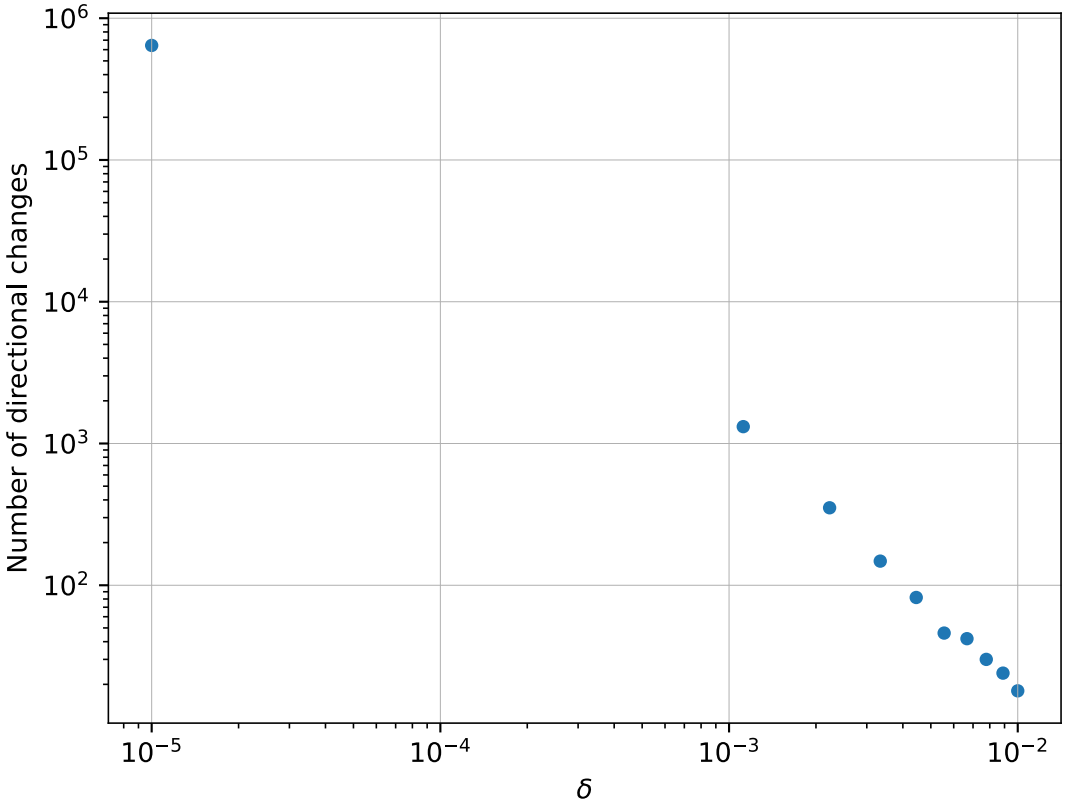


Figure 6: Number of directional changes according to the scale δ

We observe that with the log-log scale, the obtained plots seems to follow a linear decreasing. As the scale is log-log, it means that the number of directional changes follows an exponential decay when the scale δ increase.

3 Intrinsic Time Return

The purpose of this exercise is to investigate how measuring returns in intrinsic time (i.e. using directional change events as time markers) changes the statistical properties of asset returns relative to the standard physical-time approach.

1. Use the provided EUR/USD tick-by-tick dataset. Compute the mid-price as the average of bid and ask.

```
mid_price = (asks + bids) / 2
```

2. Using $\delta = 0.001$ for directional changes:

   - Compute the returns between successive directional change events (intrinsic returns).

```
def returns(ts: np.ndarray, N: int = 1) -> np.ndarray:
    result = np.full(ts.size, np.nan)
    result[N:] = (ts[N:] - ts[:-N]) / ts[:-N]
```

```
    return result


idx, changes = directional_changes(mid_price, delta=0.001)
directional_changes_returns = returns(changes)
```

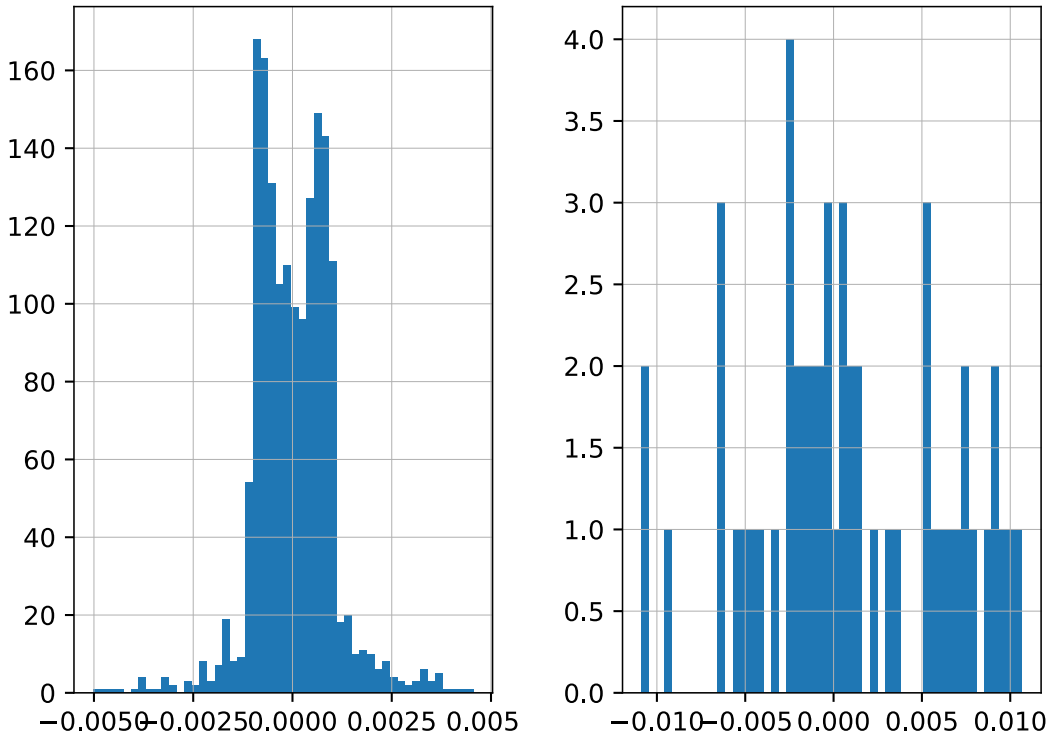
- Compute the returns over fixed physical time intervals (e.g., daily returns) for comparison.

```
days = np.array([d.date() for d in dates])
days_idx = np.unique(days, return_index=True)[1]

daily_returns = returns(mid_price[days_idx])
```

- Plot histograms of both sets of returns. Compare statistical properties (mean, standard deviation, skewness, kurtosis) of intrinsic versus physical-time returns, and discuss how the use of intrinsic time might provide different insights into market dynamics.

```
dc = directional_changes_returns[~np.isnan(directional_changes_returns)]
dr = daily_returns[~np.isnan(daily_returns)]
plt.figure()
plt.suptitle("Histograms of sets of returns")
plt.subplot(1, 2, 1)
plt.hist(dc, bins=50)
plt.title("Directional changes returns")
plt.subplot(1, 2, 2)
plt.hist(dr, bins=50)
plt.title("Daily returns")
plt.show()

from scipy.stats import skew, kurtosis

print("============================")
print("Statistical properties")
print("============================\n")
print("Directional changes returns")
print("----------------------------")
print(f"mean:               {np.mean(dc)}")
print(f"standard deviation: {np.std(dc)}")
print(f"skewness:           {skew(dc)}")
print(f"kurtosis:           {kurtosis(dc)}\n")
print("Daily returns")
print("----------------------------")
print(f"mean:               {np.mean(dr)}")
print(f"standard deviation: {np.std(dr)}")
print(f"skewness:           {skew(dr)}")
print(f"kurtosis:           {kurtosis(dr)}\n")
```

```
============================
Statistical properties
============================

Directional changes returns
---------------------------
mean:               1.8645970976510677e-05
standard deviation: 0.0009991117738714823
skewness:           0.0833892181052089
kurtosis:           3.1749828434235416

Daily returns
---------------------------
mean:               0.0007660356533830239
standard deviation: 0.005445709015972771
skewness:           -0.021417905828168927
kurtosis:           -0.671072606426844
```



(a) Directional changes returns (b) Daily returns

Figure 7: Histograms of sets of returns

The obtained mean of both measures of time are similarly around zeros.

However, the statistical propreties are very different for this two measures of time. Indeed, if we look at the Figure 7a, the bins of the histogram are well grouped around the mean whereas for Figure 7b, the bins are sparsly distributed over the space.

The standard deviation shows this clearly: in case of directional changes returns, the standard deviation is around 0.001 whereas for daily returns, the standard deviation is around 0.005, so 5 times bigger than for the directional changes.

The skewness for the daily returns is closer to zeros than for the directional changes returns, meaning that the daily return is more symetric than the directional changes return.

Finally, the kurtosis of the directional changes returns is bigger than zeros meaning the distribution is more sharp with fat tail whereas in case of daily returns, the kurtosis is less than zeros, meaning more smooth with more uniform data.

4 Crossover Strategy Simulation

In this exercise, you will have to design and backtest a simple moving average crossover trading strategy using real EUR/USD data. The strategy uses two moving averages, a fast and a slow one, to generate buy and sell signals.

1. Compute two exponential moving averages (EMA): a fast one with a short window (e.g., $\alpha_f = 2/(N_f + 1)$ with $N_f = 20$) and a slow one with a longer window (e.g., $\alpha_s = 2/(N_s + 1)$ with $N_s = 2000$).

```
mid_price = (asks + bids) / 2

N_f = 100
N_s = 2000

fast_EMA = EMA(mid_price, alpha=2 / (N_f + 1))
slow_EMA = EMA(mid_price, alpha=2 / (N_s + 1))
```

2. Define a **buy signal** when the fast EMA crosses above the slow EMA and a **sell signal** when it crosses below.

```
def buy_signal(fast: np.ndarray, slow: np.ndarray) -> np.ndarray:
    diff = fast - slow
    buy = np.zeros_like(diff).astype(bool)
    sell = np.zeros_like(diff).astype(bool)
    buy[1:] = (diff[1:] > 0) & (diff[:-1] <= 0)
    sell[1:] = (diff[1:] < 0) & (diff[:-1] >= 0)
    idx = np.arange(buy.size).astype(int)
    return idx[buy], idx[sell]
```

3. Simulate trading over the available data. For simplicity, assume you buy one unit on a buy signal and sell (or short) one unit on a sell signal.

```
buy, sell = buy_signal(fast_EMA, slow_EMA)
```

4. Plot the mid-price with the fast and slow EMAs, and indicate the buy/sell points.

```
def selection(dates: list, day_number: int = 0) -> list:
    first_day = dates[0].date() + datetime.timedelta(days=1)
```

```
    result = []
    for i in range(len(dates)):
        if dates[i].date() == first_day:
            result.append(i)
    return np.array(result).astype(int)


day_idx = selection(dates)

day = dates[day_idx]
fEMA = fast_EMA[day_idx]
sEMA = slow_EMA[day_idx]

day_buy, day_sell = buy_signal(fEMA, sEMA)

plt.figure()
plt.title("Visualization of buy signal")
plt.plot(day, fEMA, label="fast EMA")
plt.plot(day, sEMA, label="slow EMA")
# plt.plot(day, mid_price[day_idx], label="mid-price")
plt.scatter(day[day_buy], fEMA[day_buy], label="Buy signal",
color="magenta")
plt.scatter(day[day_sell], fEMA[day_sell], label="Sell signal",
color="cyan")
plt.gca().xaxis.set_major_locator(mdates.DayLocator())
plt.gca().xaxis.set_minor_locator(mdates.HourLocator())
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%b %d %Y"))
plt.gcf().autofmt_xdate()
plt.ylabel("Price")
plt.legend()
plt.show()
```

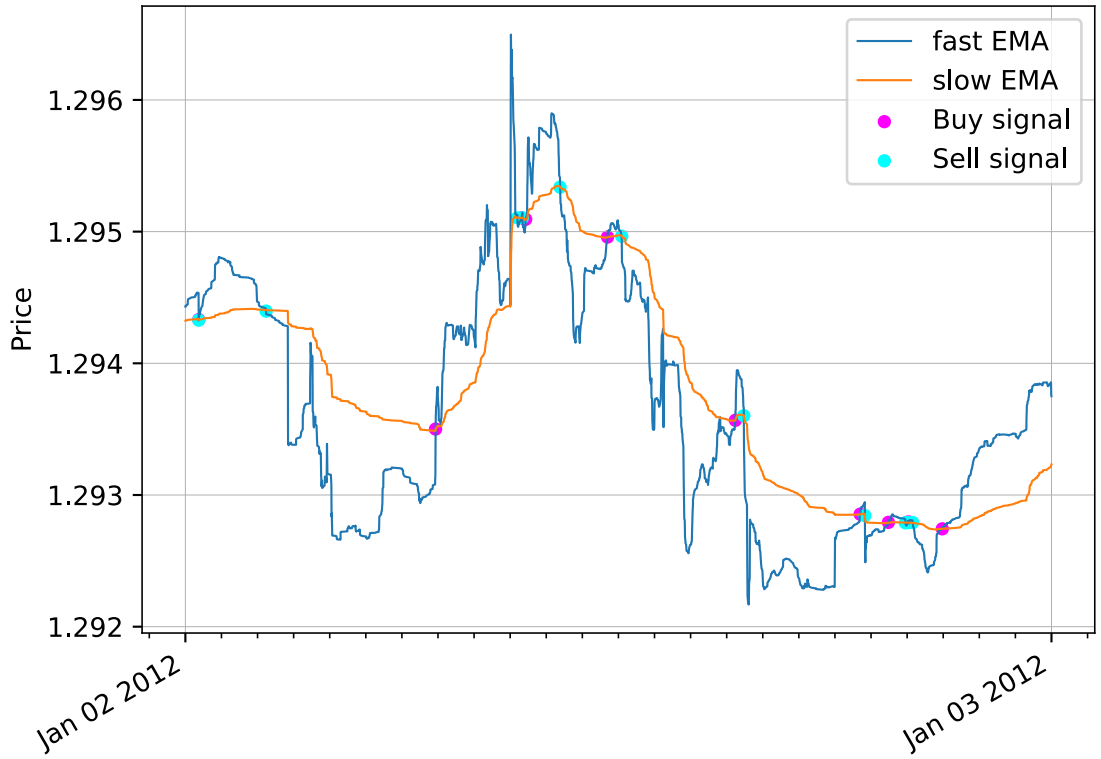


Figure 8: Visualization of buy signal

5. Discuss the strategy's effectiveness and potential limitations/improvements, for example try resampling the data (e.g., to 5-minute intervals) to reduce the number of signals: would it smooth out some of the noise? What do you observe?

   Personnaly, I think that this strategy is not so good. Indeed, the fast EMA will capture all the small variations whereas the slow EMA will capture main variations. Then, the idea is to buy and sell according to small variations, which is a good idea. Indeed, just before the price goes up, we buy, then we sell just before the price goes down. In this way, we try to take best part of the market to make some benefits. But the benefits will follow the slow EMA curve, so in fact, we don't make a lot of benefits since the slow EMA tend to be relatively uniform.

   So the idea is good, however this idea is not fully exploited. For instance, to improve the method, instead of selling when the fast EMA crosses the slow EMA, which correspond approximately to no big benefits, we can sell when the price stop growing after we buy to make more benefits. Indeed, when we will sell, the price will be bigger than if we sell when the fast EMA crosses the slow EMA.